

Multimodal Stock Prediction: Using Sentiment and Historical Data to Predict Stock Price Direction



WPI

A Paper Submitted to:
Professor Fabricio Murai

Authors:

Joshua Andrade, Preet Patel, Kobe Richards and Atharva Tiwari

Abstract

Predicting stock price direction is a complex challenge due to the market's sensitivity to both historical trends and real-time sentiment. In this paper, we present a multi-modal deep learning framework that integrates sentiment analysis from financial news with historical stock data to improve stock price direction prediction. We begin by developing individual models: FinBERT with both regression and classification heads for sentiment analysis, and multiple architectures—including a Naive Forecast, Bidirectional LSTM, and Multi-Task Transformer—for historical price prediction. Our final architecture, a Dual-Stream Transformer model, combines both modalities using cross-attention to capture intricate relationships between textual sentiment and time-series patterns. Evaluated on a dataset of diverse S&P 500 stocks, our hybrid model achieved state-of-the-art performance, with up to 97.81% accuracy on next-week price movement prediction. These results highlight the effectiveness of combining financial NLP with engineered technical indicators for robust and interpretable stock forecasting.

Abstract.....	1
Introduction.....	3
Recent Works.....	3
Dataset Overview & EDA.....	5
Historical Price.....	5
Sentiment Analysis.....	7
Methodology.....	10
Finbert with a Regression Head.....	10
Finbert with a Classification Head.....	11
Hyperparameter Tuning with FinBERT models.....	11
Naive Forecasting Model.....	12
Bidirectional LSTM Model.....	12
Multi-Task Transformer Model.....	13
Dual-Stream Transformer Model.....	13
Results.....	16
Finbert with a Regression Head.....	16
Finbert with a Classifier Head.....	19
Naive Forecasting Model.....	19
BiDirectional LSTM Model.....	20
Multi-Task Transformer Model.....	20
Dual-Stream Transformer Model.....	20
Conclusion and Future Steps.....	23
References.....	24

Introduction

Every investor wants to be able to predict stock behavior in order to capitalize for a profit. However, this task remains challenging for companies and day-traders due to inherent volatility, noise, and sensitivity in the market. Stock market behavior can be influenced by a variety of factors, including but not limited to macroeconomic policies, investor sentiment, and economic indicators. The unpredictability of these factors is the reason why it is very difficult to predict stock movement with accuracy.

Deep learning techniques may be a solution in uncovering underlying complex patterns in historical stock price data and investor sentiment. These models have shown better accuracy than random prediction. Traditional forecasting methods have used mainly two different methods for stock price prediction. The first approach is relying on historical stock price data, with time-series forecasting techniques like AutoRegressive Integrated Moving Average (ARIMA), Long Short-Term Memory networks (LSTMs) or other advanced networks optimized for time series data. The second approach is using financial news reports and articles to perform sentiment analysis to stock market analysis. If investors receive positive news, they are more likely to buy a company's stock, increasing the price. On the other hand, if investors receive negative news, they are more likely to sell a company's stock, decreasing the price.

Recent Works

- I. One prior study focused mainly on applying LSTM and GRU architectures to predict stock movement in the technology sector (Chang et al.). They trained their models solely on historical stock data by using Yahoo finance as the source for their dataset. Their data consisted of the standard, raw API features: Open, Close, High, Low, Adjusted Closing Price, and Volume. They downloaded this information for four of the biggest tech stocks - Apple, Amazon, Google, and Microsoft - with a timeframe ranging from January 1st, 2013 to March 30, 2022 (Chang et al.). Although the training data they used is very large, it may not capture the full picture when it comes to the stock market behavior. The dataset does not reflect a diverse mix of stock options. Apple, Amazon, Google, and Microsoft can all be considered steady stocks due to how big the companies are. They do not capture the volatility and unpredictable behavior other stocks like startups may have, which can lead to a model that is not generalizable. In addition to this, these raw features may not help the model fully capture the complex patterns in the stock market. In this study, we aim to address these weaknesses by performing exploratory data analysis (EDA) on the raw features to generate new informative features and choose a wide variety of stocks with different behaviors for a more predictive model.
- II. Another prior study used technical indicators and financial news articles to predict stock market behavior (Vargas et al.). In order to analyze financial text and determine sentiment, they used an embedding layer which takes encoded sentences as input. They

used titles of financial articles. The first step in the encoding process of the sentences was done through a Word2Vec model which generated word embeddings. Second, they took an average of all the word vectors in a title to output a sentence vector (Vargas et al.). Instead of using a simple averaging scheme for word embeddings, we will utilize a specific sentiment classifier—FinBert transformer—to extract sentiment scores from financial headlines. We aim to use transformers for sentiment analysis because they can be more powerful at capturing context and meaning in a piece of text.

In this paper, we propose a hybrid deep learning model that leverages sentiment and historical stock data to make a prediction on future stock behavior: up or down in price. First, we created two independent models. The first one predicted stock price direction based only on historical stock data. We built and tested three different architectures for this model, including a Naive Forecast, Bidirectional LSTM, and a Multi-Task Transformer (ARIMA). The second model used only sentiment from financial articles to predicted stock behavior. For this model, we built a FinBert with a classification head and one with a regression head. Following this, we took the best performing architecture for each independent model and performed a late fusion of them using a dual stream transformer as you will see in a later section.

Dataset Overview & EDA

Historical Price

We used “yfinance”, a python library that allowed us to retrieve historical stock data from Yahoo Finance. This API is widely used for financial data and analysis. This data is returned in a Pandas dataframe consisting of common stock features such as the opening price, closing price, high price, low price, volume, return, and date. We used this numerical data as training data to our model. We chose to use stocks from the S&P 500 as this index represents the largest 500 publicly traded U.S companies. The S&P 500 covers around 80 percent of the U.S market equity capitalization. To choose what stocks we wanted to use to train our model, we downloaded data from the past 3 years for all 500 stocks. In order to create a model that could generalize across different market behaviors, we chose to include a diverse mix of stocks in our training with different behaviors: steady, stagnant, and variable. To classify each trend type, we calculated the total return and volatility for each stock.

Table 1

	Total Return	Volatility
Stagnant	< 0.05	< 0.02
Steady	> 0.2	< 0.03
Variable	Other	Other

The thresholds in the table above were used to determine what stocks are stagnant, steady, or variable. After grouping the stocks into these three trend types, we picked the five most popular ones in each group. To determine popularity, we used the market cap value of the stock. The output was the following tickers:

Top 5 most popular steady stocks: AAPL, MSFT, GOOG, GOOGL, AMZN

Top 5 popular variable stocks: ABBV, ADBE, ADI, ABNB, ZTS

Top 5 popular stagnant stocks: XOM, JNJ, ACN, CMCSA, LMT

The next step in creating our dataset was to perform feature engineering on the raw features the Yahoo Finance API offers. This allowed us to generate more informative features to

use in order to create a model with more predictive power. We calculated the new technical indicators using the raw features. These new features are shown in the table below:

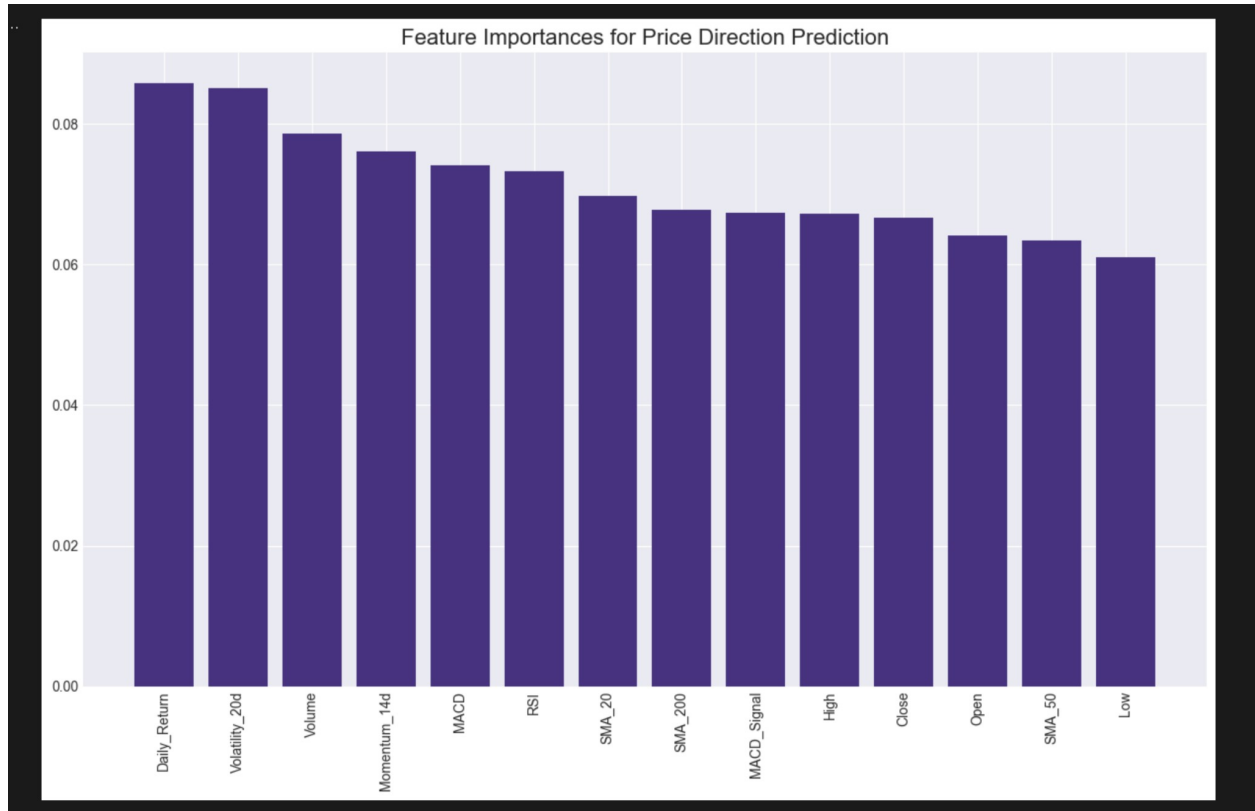
Table 2

Feature	Definition
SMA_20	Simply Moving Averages over previous 20 days
SMA_50	Simply Moving Averages over previous 50 days
SMA_200	Simply Moving Averages over previous 200 days
EMA_12	Exponential Moving Averages over previous 12 days
EMA_20	Exponential Moving Averages over previous 20 days
EMA_26	Exponential Moving Averages over previous 26 days
BB_Lower	Bollinger Bands Lower
BB_Middle	Bollinger Bands Middle
BB_High	Bollinger Bands High
RSI	Relative Strength Index
MACD	Moving Average Convergence Divergence
MACD_Signal	Moving Average Convergence Divergence Signal
MACD_Hist	Moving Average Convergence Divergence Histogram
Volume_SMA_20	Simple Moving Average over the previous 20 days on Volume
Momentum_14d	Momentum of previous 14 days

These features help the model better learn different temporal trends and dependencies in the data. We trained a random forest classifier on each of the fifteen chosen stocks with the raw and newly made features. For each stock, we plotted a graph to determine which features were most important in predicting its movement. For example, the feature importance graph for Apple (AAPL) is shown below:

Figure 1

Feature Importance for price direction prediction.



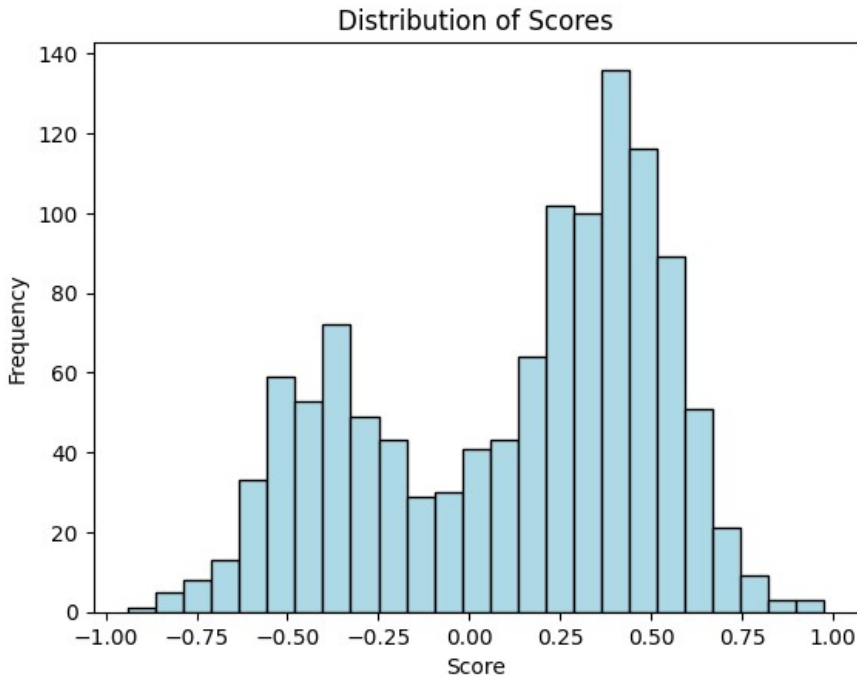
We analyzed the graph for each stock to find the commonality features between them. The common features we chose to use were: Return, SMA over different number of days, RSI, MACD, MACD_Signal, and Volatility.

Sentiment Analysis

For the sentiment analysis portion of the project we used 2 main sources. First, for FinBERT with a regression multi-layer perceptron the FiQA dataset was used. The Financial Question Answering (FiQA) dataset is a corpus curated to support research in financial natural language processing (NLP). This is a smaller dataset with 1174 entries and is constructed from financial news headlines, microblogs and articles that were annotated by experts in the field of finance and trading. Each sample in this dataset is labeled with a sentiment score ranging from -1 (very negative) to 1 (very positive). As part of understanding the dataset we conducted an EDA of the dataset to understand it better. First, we look at the distribution of the sentiment scores, the figure below showcases the distribution of the data. This histogram illustrates the distribution of prediction scores, the distribution appears to have a bimodal shape with 2 prominent peaks, one centered around -0.4 and another around 0.4. The dip near score = 0, indicates that the model rarely outputs ambiguous or neutral predictions, which is desirable if the model is confident in its outputs.

Figure 2

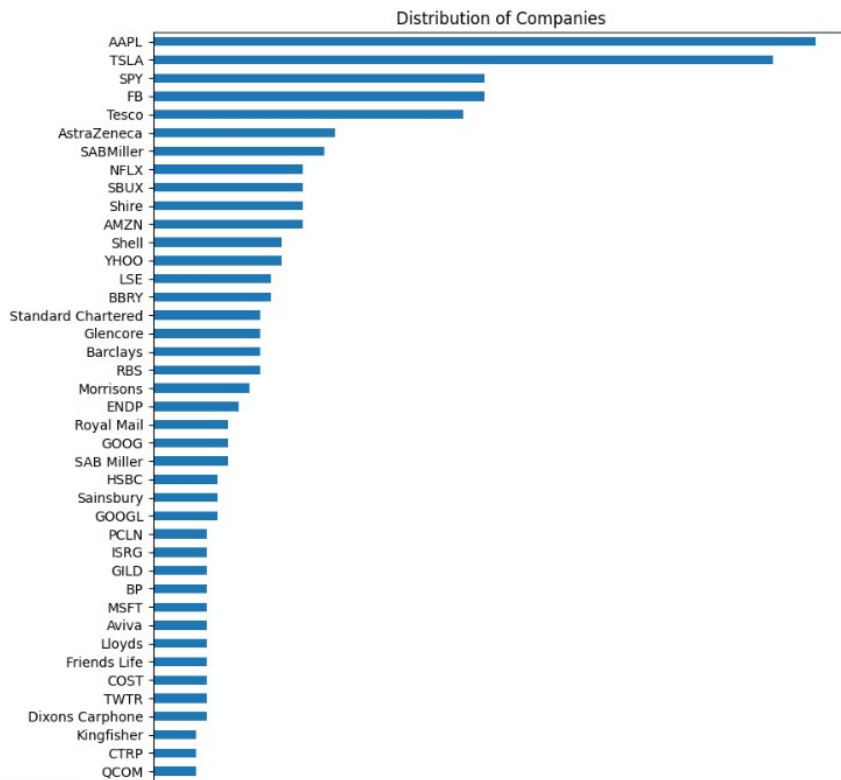
This figure shows the distribution of scores from the FiQA dataset.



Naturally, another consideration that we had to make was to understand the distribution of companies that were present in the dataset. This is important because we want to understand the types of stocks that are present in the dataset and to make sure that the models that we are training on this dataset are capturing a wide-variety of sentiments. There were over 50 companies present in the dataset and a snippet of the companies are shown in the figure below. Although it might not be seen in the image below, it is clear that most of the data in this dataset is from stocks that are in tech-sector indicating that the model may be better geared towards using sentiment to predict stock movement in the tech sector.

Figure 3

This figure represents the distribution of companies present in the FiQA dataset.



The dataset mentioned above was primarily used to train the regression head version of the FinBERT. For the classification head and for the hybrid model the NewsAPI was used to generate datasets. This API provides access to real-time and historical news from thousands of global resources. With a free account we were limited to accessing data that was up-to a month old. When we were training the classification head and the dual-stream transformer model the dataset was generated through making calls to this API for certain stocks, which were decided using the approach discussed in the *Historical Price* section. This was the approach used to gather the headlines that we would use for the dataset. To retrieve the movement of the stock price for the stock for the next day, the yahoo finance API was used to retrieve the prices then a calculation of closing price from the present day to the next was used to generate the movement of the particular stock. In sum, for the sentiment analysis portion of the project, there were multiple sources of data that were used depending on the model, the FiQA dataset was used primarily for the regression head, the NewsAPI paired with feature engineering from finance was used for the dual-stream model and the classification head on FinBERT.

Methodology

Finbert with a Regression Head

FinBERT with a Regression head was implemented as a means to generate a sentiment score for a set of news headlines. *What is FinBERT?* FinBERT is a financial domain model that is based on BERT (Bidirectional Encoder Representation from Transformers) architecture, and is fine-tuned on financial specific data using Masked-Language Modeling (MLM) and Next-Sentence Prediction (NSP). Unlike general purpose language models, FinBERT successfully captures the language and context that is unique to financial language; this makes this model effective in extracting sentiment from market-related news (Araci, 2019).

For the regression head, we take this base model and attach a regression head onto it. This regression head follows the architecture shown in the figure below. It uses a linear layer with a ReLU activation function, followed by another linear layer with a tanh activation function to output a value between -1 and 1. It is important to note that alternate model architectures were used in the selection process but they did not provide any improvements in MSE. The final modified model takes the 768 [CLS] token that contains a representation of the sentiment pertaining to a headline and then passes it through an MLP with a single node in the final layer that produces a regression score.

Figure 4

```
#Define the regressor head
self.regressor = nn.Sequential(
    nn.Linear(768, 256),
    nn.ReLU(),
    nn.Linear(256, 1),
    nn.Tanh()
)
```

This figure shows the model architecture that was used for the regression head.

After training, came the inference phase. To accomplish this task we gathered headlines from the NewsAPI, after generating the sentiment scores using the model outlined above, the sentiment scores were plotted against the actual stock movement to see how well the direction of the stock was predicted. To measure accuracy, we used a sliding-window correlation score to judge how close the sentiment scores followed the stock price. Although this model was not trained directly to be used for a classification task, it served as an alternate approach to see if we can get increased accuracy on predicting the movement of stock.

Finbert with a Classification Head

FinBERT with a classifier head was implemented to see the strength of the correlation between the headline and the direction of the stock price. The direction of the stock price was determined by comparing the closing price of the stock on the day of the article's publication to the next day. It's important to note that the model was supposed to be a classifier model between classifying the stock price direction as going up, going down, and staying the same, but was later made into a binary classifier since there were no examples of stock price staying the same in our dataset.

For the classifier head, the base BERT model is taken and a classifier head is attached to it. This classifier head follows the architecture shown in the figure below. It uses a simple layer that takes the 768 [CLS] token, which contains a representation of the sentiment related to a headline and outputs a logit, an unnormalized prediction score, which is then put through a sigmoid activation function to give the prediction. It is equivalent to `nn.Linear(768, 1)`, `nn.Sigmoid` in Pytorch. Alternative classifier heads were looked at, such as a deeper MLP, but they performed worse than the simple one-layer head used. The loss function used was binary cross-entropy with logits, but weighted to account for the fact that the dataset was heavily imbalanced, with only 25% of the headlines leading to lower stock prices.

Figure 5

This figure shows the model architecture that was used for the classifier head.

```
self.classifier = nn.Linear(self.bert.config.hidden_size, 1)
```

Hyperparameter Tuning with FinBERT models

Both the FinBERT with a regression head, and FinBERT with a classifier head had their hyperparameters tuned with Optuna. Optuna is a framework for optimizing hyperparameters, but unlike traditional methods like grid search, Optuna uses a form of Bayesian optimization to tune hyperparameters called tree-structured parzen estimator (TPE). TPE uses the conditional distribution of hyperparameters from a given objective value, as well as the marginal distribution of objective values. The marginal distribution is used to form a threshold which is then used with the conditional distribution to create two different densities. These densities are distributions where one shows hyperparameter configurations that perform well, based on the threshold, and the other shows hyperparameter configurations that perform poorly based on the threshold. What TPE does is tries to maximize the ratio of the good hyperparameter distribution to the bad one, ultimately trying to sample a hyperparameter configuration that appears often in good trials, and is uncommon in bad trials. TPE uses a tree-like structure for its search space, which allows for it to be more efficient and not waste as many trials, as well as examine conditional relationships between hyperparameters. It can also have mixed-types, meaning that the hyperparameter space it searches for can be a discrete or continuous range, as well as be categorical, which means specific discrete values.

Compared to other hyperparameter tuning methods such as grid search, TPE scales quite well, since it builds a better understanding of the search space with more trials, allowing it to converge on an optimal combination. This applies with larger amounts of hyperparameters, hyperparameters with more complex relationships with each other, and larger spaces of hyperparameters. It is also capable of searching continuous ranges, unlike grid search which requires a categorical list of values. It is also faster than grid search, unless you have very few hyperparameters with a small list of values. It is more effective at tuning hyperparameters with little to no downsides, and it was the method of choice for hyperparameter tuning for both FinBERT models, as well as for the autoencoder that is discussed later.

Naive Forecasting Model

A Naive Forecasting model was implemented as a simple, rule-based baseline to establish a point of comparison for evaluating stock price direction prediction. This model operates without the need for training nor any parameter optimization by the user, making predictions solely based on the assumption that recent price trends are likely to persist. While prediction is done stock-wise for each input day, the model compares the relation between the closing price of the stock on that particular day and its closing price of the preceding day. If the price has increased, the model predicts an upward movement (class label 1) for the following day; otherwise, a downward or unchanged movement is predicted (class label 0). This logic is extended to multi-horizon by considering the comparison between the current price with the price at a future offset (e.g., 5 days ahead for next week and 20 days ahead for next month).

Despite its simplicity, the Naive Forecasting model acts as a benchmark performance standard in financial modeling. It is a heuristic common to finance—that momentum tends to continue for the short-term—and is thus a signal that knows a lot despite its apparent lack of sophistication. Without using historical context or technical indicators other than the singular previous price change, any modeling approach seeking to outperform this baseline has to learn and discern increasing complexity in temporal patterns or find a richer combination of features. Therefore, the Naive forecasting model provides a benchmark value for more complex machine learning methods to be measured against.

Bidirectional LSTM Model

A Bidirectional LSTM model was created to classify stock price movement based only on historical stock data. The training data for this model consisted of the features we engineered with the fifteen different chosen stocks. We added a new label column named ‘Price Change’, which represented if the closing price for that day increased or decreased from the previous day’s closing price. A 1 reflected an increase in price while a 0 reflected a decrease in price. This data was then transformed into different 30 day rolling sequences using a sliding window technique for each stock. Each 30 day sequence had its own label assigned to it, which was the next day’s price change label. The data was split into a training set and validation set with an 80:20 ratio.

After the data was processed and normalized, each sequence was then fed into the LSTM as input. The architecture consisted of two hidden layers with 128 units and 64 units respectively. Each layer was followed up by a dropout layer of 0.3. This output was then sent to a ReLU activation layer and a sigmoid activation for binary classification. The learning optimizer we chose was Adam. We chose to use a Bidirectional LSTM in order to leverage its strengths in capturing temporal patterns in data by analyzing past and future contexts. This allows the model to extract richer information from features over a regular LSTM.

Multi-Task Transformer Model

A Multi-Task Transformer model was developed to predict stock price direction across three time horizons—next day, next week, and next month—using only historical market data. For each of the 15 selected stocks, a three-year window of daily historical data was collected and processed into 30-day rolling sequences. Each sequence served as a single input sample, incorporating a rich set of engineered technical indicators such as daily returns, moving averages (MA-7, MA-14, MA-30), relative strength index (RSI), MACD and its signal line, closing price, and 30-day rolling volatility. These features were chosen for financial relevance and then verified by a feature-importance analysis that proved their high predictive value for a set of stocks.

Each 30-day sequence was standardized and sent to a linear embedding layer so the data could be projected into a higher-dimensional space required by attention mechanisms. At the heart of the model is a four-layer Transformer encoder with four self-attention heads per layer. This powerful encoder is able to search for temporal dependencies in the input sequence and allow the model to dynamically weight importance on certain days that may govern price movement in the future. After encoding, the output is pooled by mean across the sequence dimension to form a fixed-length representation. This mean-pooled vector is fed into three parallel output heads, each being a fully connected layer, predicting up-or-down binary price direction for the next day, next week, and next month, respectively.

This setting benefits from multi-task learning since it forces a model to develop certain shared representations, which in turn aid in generalization. Not only efficient inference for multiple time horizons would be promoted by such a design, but also simultaneous learning of short-term and long-term market dynamics would be enabled within a unified architecture. This results in a flexible and robust forecasting model capable of harnessing the temporal and technical patterns embodied by financial time series data.

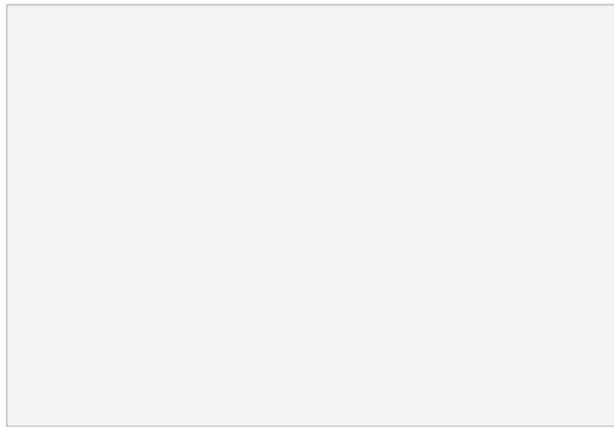
Dual-Stream Transformer Model

A dual-stream transformer model was used as a means to combine the modalities and have them attend to one another. An overview of the model is seen in the figure below. As it can be seen there are 2 inputs, which is a 128-dimensional vector from the multi-task transformer and the 128-dimensional vector from FinBERT. These inputs go into a cross-attention layer, the purpose of this layer is so that the different modalities can attend to one another. This means that

the model can “understand” how changes in the stock price should influence the interpretation of the sentiment and vice versa. Additionally, the goal of this cross-attention layer is to help the model make more informed decisions by combining the contextual information from both modalities. The cross-attention layer serves as a critical mechanism for fusing the two distinct modalities of data.

Figure 6

This figure showcases in a simple diagram the model architecture that was used for the dual-stream transformer model.



One important part about the cross-attention layer of the model is that due to computational restrictions we used dot-product attention and for this the dimensions of the inputs need to be the same. This is because for a valid dot-product to occur between two vectors, they must have the same dimensionality. As a result, we needed to downsample the 768-dimensional [CLS] token that we use from finBERT. To achieve this an autoencoder was trained to downsample the embeddings to be a 128-dimensional vector. In order to train this model headlines were gathered and their embeddings were generated and the autoencoder learned to reconstruct these embeddings. For the purposes of the dual-stream transformer model, only the encoder portion was used to generate the 128-dimensional vectors. The final portion of the model was a classification head that would be used to classify the movement of the stock price for the next day and for the next week. Due to limitations on the amount of data that we had we were restricted to these two horizons to predict from.

To train this model, we constructed a dataset that included engineered financial features such as daily returns, simple moving averages (SMA-20, SMA-50, SMA-200), exponential moving averages (EMA-12, EMA-26), relative strength index (RSI), MACD and its signal line, rolling volatility, and trading volume. These features were concatenated with the 128-dimensional embeddings generated from FinBERT and compressed using an autoencoder, resulting in the final input vectors used to train the model. Given the presence of class imbalance—primarily due to limited news article coverage for certain stocks on some days—we

opted to use Focal Loss as the training loss function. Focal Loss is a variant of cross-entropy loss that reduces the relative loss for well-classified examples and puts more focus on hard, misclassified examples, making it particularly effective in imbalanced classification scenarios (Lin et al., 2017). The model was trained with early stopping to prevent overfitting, and Optuna was employed for hyperparameter optimization. This automated process helped identify the optimal combination of learning rate, weight decay, and other training parameters for best performance.

In summary, this dual-stream transformer architecture harnesses both numerical and textual modalities through cross-attention. This enables the model to capture more nuanced interactions between stock and sentiment trends. By integrating engineered price-based features with rich sentiment embeddings, the model was positioned to make more context-aware predictions. Although we were limited by the availability of sentiment data, this approach demonstrates a potentially scalable framework for financial forecasting.

Results

Table 3

Table showcasing the results for the models used throughout this project

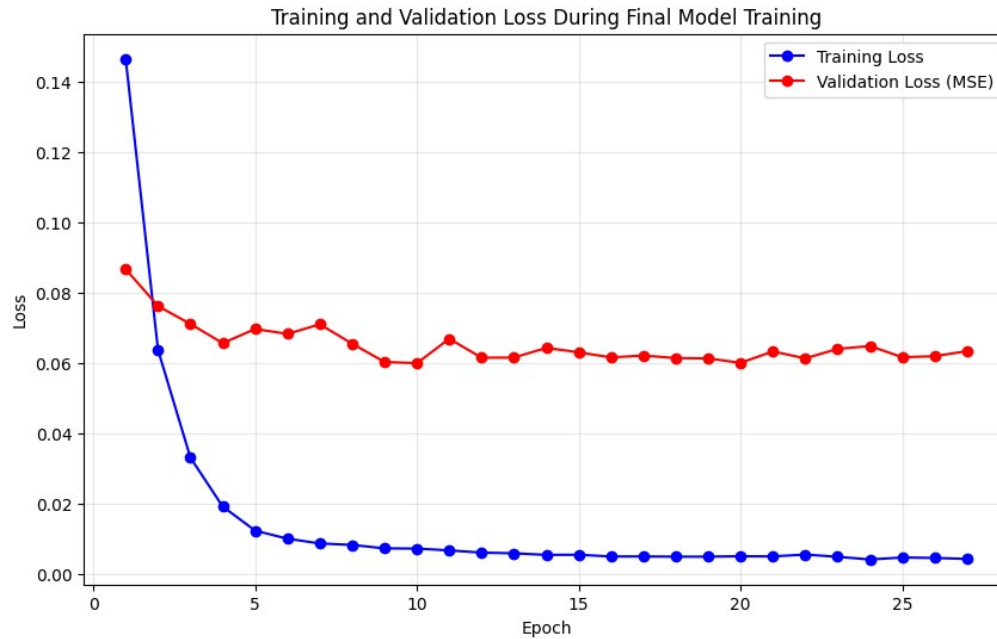
Model		Accuracy	Precision	Recall	F1-Score
FinBERT (Reg head)		70%	65%	57.14%	60.82%
FinBERT (Class head)		77.53%	82.26%	89.47%	85.71%
Naive Forecasting		51.73%	54.92%	54.92%	54.92%
Bidirectional LSTM		49.96%	53.67%	13.95%	22.14%
Multi-Task Transformer	Next Day	55.41%	55.25%	56.46%	55.85%
	Next Week	71.50%	71.71%	75.89%	73.74%
	Next Month	85.90%	87.45%	85.47%	86.45%
Dual-Stream Transformer	Next Day	92.70%	93.03%	93.03%	93.03%
	Next Week	97.81%	98.03%	97.27%	97.65%

Finbert with a Regression Head

For FinBERT with a regression head we used 10-fold cross-validation to select the best performing model and its hyperparameters. The primary goals of this step was to ensure that the model's performance was not an artifact of any particular data split, evaluate its consistency across data partitions and to select the best-performing hyperparameters for this task. We then re-trained on the entire dataset, this is done because the re-trained model having seen all the data is likely to generalize better when applied to new, unseen financial texts during real-world inference or integration into a larger prediction pipeline. The results of this training approach are seen in the figure below. Based on the figure below, it can be concluded that the model learned robust sentiment representations without overfitting, as demonstrated by the stable validation loss even after training loss reached a near-zero plateau. It is important to note that the validation loss did remain consistently above the training loss through training, reflecting the expected generalization gap between seen and unseen data. This gap remains stable over epochs, suggesting that the model achieved a balance between fit and generalization.

Figure 7

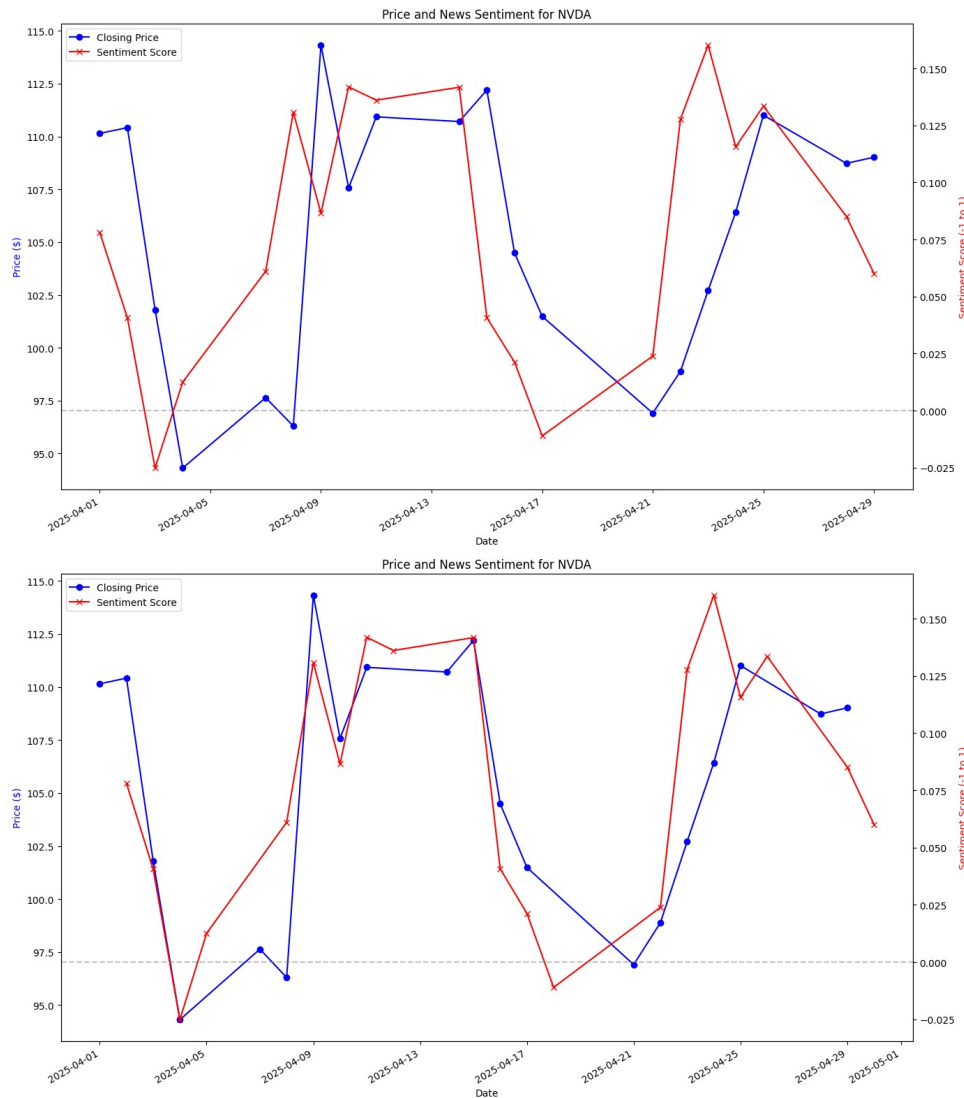
The graph showcases the training loss vs. the validation loss during the final re-training of FinBERT with a Regression head.



After the model was trained came the inference task, which was being able to use news headlines to judge whether the stock price goes up or down. To accomplish this we used this model to generate the sentiment scores which were then averaged across a date. This was done because there were multiple headlines per day and it would be a good way to understand the sentiment for one-day. After having accomplished this task, these results were then plotted against the movement of the actual stock price, an example can be seen in the figure below. As seen there appears to be some alignment between sentiment score spikes, but it follows a lead-lag behavior. Sentiment sometimes leads the price-changes by 1-2 days. Other times, sentiment lags slightly behind sharp market movements. To measure the accuracy of these predictions we used correlation but with a sliding window approach to see how the correlation evolves across the time dimension. The lead-lag behavior reduces this value to an average of 0.39, however, once we shift the data the sliding-window correlation goes up to an average of 0.652. It is important to note that this for the NVIDIA stock specifically and these values change for the type of stock.

Figure 8

*Top: the graph on top shows the original sentiment scores plotted against the stock prices.
Bottom: the graph shows the shifted sentiment scores plotted against the stock prices.*

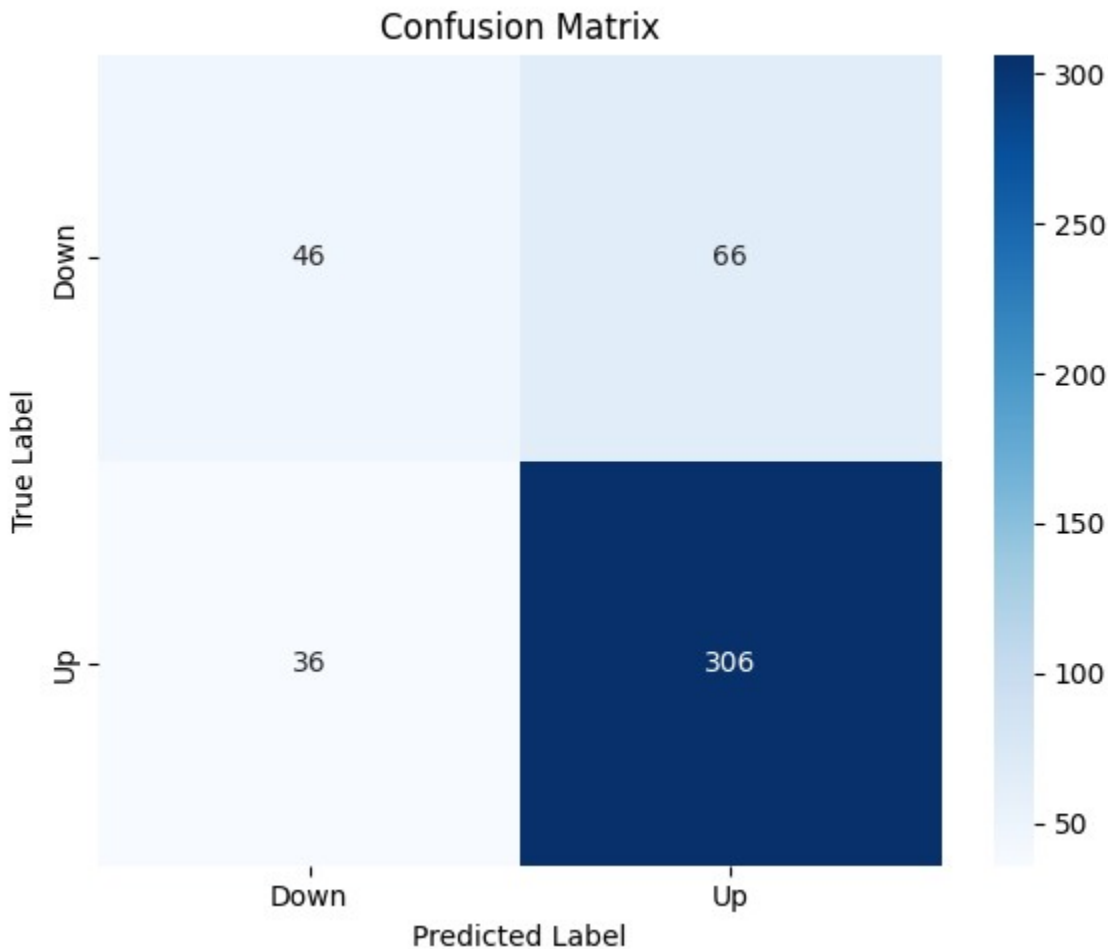


Additionally, we were able to compare this model with the other models we engineered the accuracy, precision, recall and f1-score metrics. We accomplished this by calculating the change in closing price to determine if the stock price goes up or down, the same approach was used for the sentiment (but with sentiment scores). After doing this we averaged our results across the stocks that we looked at to achieve the values listed in the results table above. In sum, the results of this model indicate that using sentiment alone cannot be used to predict the movement of the stock price and there need to be additional features that need to be considered to increase the accuracy of these predictions.

Finbert with a Classifier Head

Figure 9

Confusion matrix for FinBERT with a classifier head



FinBERT with a classifier head was able to get 77% accuracy, 82% precision, 89% recall, and 85% f1-score. However, these aren't necessarily indicators of strong performance. With how imbalanced the dataset was, being 75% increase in stock price, and 25% decrease in stock price, an accuracy of 77%, even with weighted loss to attempt to account for the imbalance, means that the model is doing just a little better than if it always predicted increase in stock price. The confusion matrix reveals the flaw in the imbalanced dataset, and how it results in a very poor ability to accurately predict a stock price decrease.

Naive Forecasting Model

The Naive Forecasting model, which assumes that the direction of the next price movement will follow the direction of the most recent movement, achieved an accuracy of 52.73% with similarly low precision, recall, and F1-score values of 54.92%. These results reflect the simplicity of the model—it does not learn from patterns or incorporate any technical

indicators, relying solely on short-term momentum. While marginally better than random guessing in a binary classification setting, its limitations become clear when compared to more complex architectures. Despite this, it serves as a useful baseline; any model performing better than this can be considered to extract meaningful patterns from the data.

Bidirectional LSTM Model

We evaluated the Bidirectional LSTM with four different metrics: accuracy, precision, recall, and F1 score. The results can be seen in the table above. The model was not able to predict stock behavior for the next day too well. This may be because of improper class weights where one class was favored by the model more than another, even after accounting for possible weight imbalance in the model. The model clearly struggled in learning different temporal patterns in the training data. It underperformed the multi-task transformer by around 5-6 percent. Due to this, we decided to use the multi-task transformer for the full hybrid model.

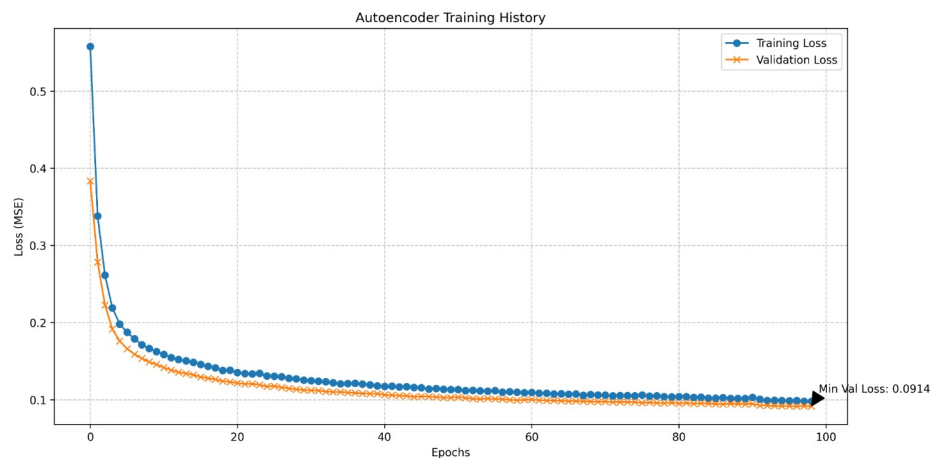
Multi-Task Transformer Model

Against all time horizons, the Multi-Task Transformer outperformed the naive baseline, thereby proving its ability to learn temporal dependencies and complex market behavior. Accuracy values of 55.41%, 71.50%, and 85.90% were reported for next-day, next-week, and next-month predictions, respectively. Similar enhancements were seen in precision, recall, and F1-score as the time window increased, with the F1-score peaking at 86.45% for the monthly horizon. These numbers indicate that the model was able to better capture longer-term trends, facilitated by the self-attention mechanism of the transformer and the engineered financial features. Multi-task learning also helped by inducing a better generalization as the training process involved jointly learning several tasks of different time horizons, thus improving the model's robustness and consistency across forecasts.

Dual-Stream Transformer Model

When training the dual-stream transformer model, we used an autoencoder to be able to reduce the embedding dimension for the [CLS] token from being a 768-dimensional vector to a 128-dimensional vector with minimal reconstruction loss. This was needed as we were using dot-product attention and needed the dimensions to be the same and to ensure efficient fusion. This compact representation preserved the integrity of the original features and improved the computational efficiency and generalization capability of the hybrid model by reducing the complexity of the input space used in the cross-attention mechanism. The results of the training loss for the autoencoder are seen in the figure below. It can be seen that throughout the training process, the validation loss remains lower than the training loss, this paired with the fact that the training loss and validation loss converge shows that the model is learning effectively without overfitting.

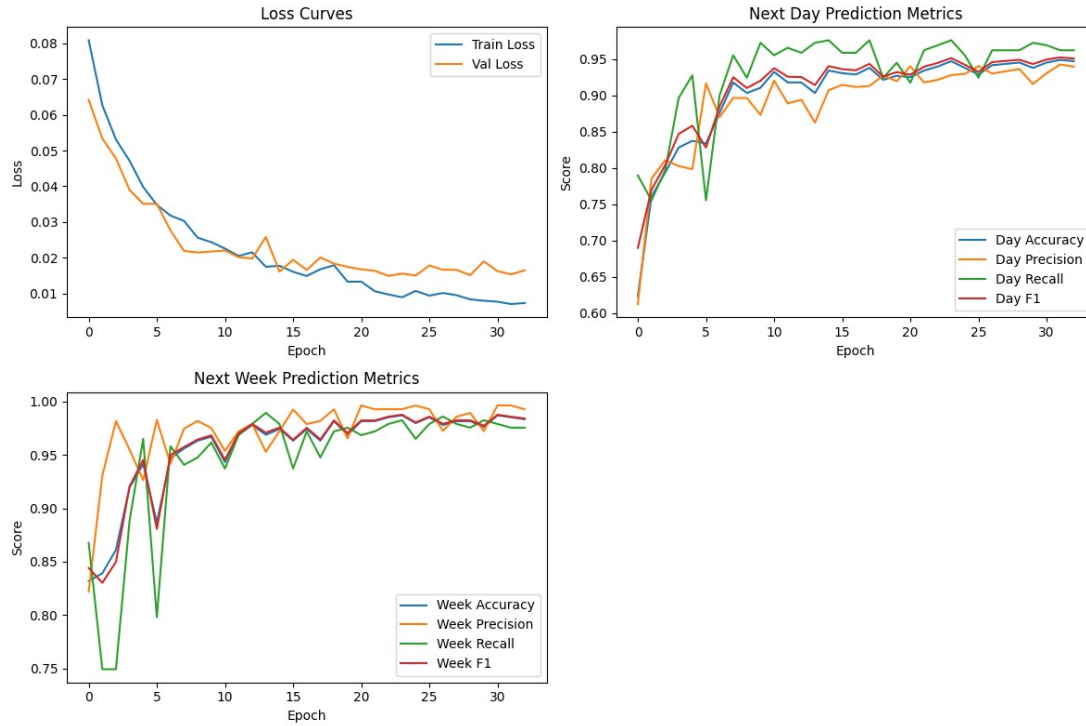
Figure 10
Training Loss vs. Validation loss for autoencoder training.



After the autoencoder has been trained the model was used to generate the 128-dimensional embeddings and concatenated onto the dataset that was to be used in training the dual-stream transformer. For training this hybrid model we used focal loss, with early stopping based on validation loss and regularization through dropout and weight decay. On the next-day prediction task, this model achieved an accuracy of 92.70%, with precision, recall and F1-score all at 93.03%. These sets of results indicate the model's ability to consistently identify short-term directional changes. More importantly, the precision-recall parity suggests the model was well-calibrated and did not overfit to a particular class. For the next-week prediction task, performance improved, with an accuracy of 97.81%, precision of 98.03%, recall of 97.27%, and an F1-score of 97.65%. The high scores in the weekly horizon indicates that the hybrid model may be more proficient at capturing longer-term dependencies and trends that emerge over time. It is important to note that we did not look at the next-month horizon as we were unable to gather enough data from the news headlines to be able to generate a robust dataset that goes as far back as a year. Below, the figure contains information about the loss during training the prediction metrics for the next-day and next-week horizons.

Figure 11

This figure shows the loss curves during training, next-day prediction metrics and next-week prediction metrics.



These results show strong empirical support for the dual-stream transformer approach. The multi-modal fusion of sentiment and price data allowed the model to extract patterns that are not seen when either modality is used in isolation within their respective models. Moreover, the high F1-scores in both tasks showcase the model's robustness and utility for real-world stock forecasting applications. The dual-stream transformer provides an encouraging framework for future research in multimodal stock prediction.

Conclusion and Future Steps

Our model demonstrated an accurate way to predict whether a stock's price will increase or decrease in the future. We were able to do this with a different approach than prior methods explained earlier in the paper. This model capitalizes on weaknesses we saw in prior approaches through advanced feature engineering and a more advanced NLP architecture. Our model is able to predict stock price movement with much greater accuracy than the best model created by Vargas et al with a 56.84% accuracy. Although the results can not be directly compared with work Chang et al. done due to our model being a binary classification, we can see that our more advanced features paired with a multi-modal approach with sentiment still led to a very accurate model. In the future, we can improve our hybrid model by incorporating more data by tracking U.S politician trades or earning call transcripts. This will allow the sentiment portion of the model to have more access to relevant data that can be used to predict the movement of the stock. Politician trades can offer valuable insights for predicting stock movements because elected officials often have access to non-public information or early awareness of upcoming legislation, regulatory shifts, and policy changes that can significantly impact specific sectors or companies.

We had several different limitations in our model. The first was the timespan we had to conduct this work. With more time, we could test more architectures for each of our independent models or fusion techniques such as an early fusion. We also could include interpretive frameworks like SHAP (SHapley Additive exPlanations) or LIME (Local Interpretable Model-agnostic Explanations) to better understand the decision-making process of our model. These tools help explain individual predictions by identifying which input features (e.g., specific sentiment scores or technical indicators) had the most influence on the model's output. This will allow us to analyze whether the model relies more heavily on sentiment or historical price data when forecasting stock movement, adding transparency and interpretability to our results. Another limitation was that we were only able to get data for sentiment analysis for the past 25 days or less. This restricted the amount of data available for training.

References

- Araci, D. (2019, August 27). *FinBERT: Financial Sentiment Analysis with Pre-trained Language Models*. arXiv.org. <https://arxiv.org/abs/1908.10063>
- Chang, V., Xu, Q. A., Chidozie, A., & Wang, H. (2024). Predicting Economic Trends and Stock Market Prices with Deep Learning and Advanced Machine Learning Techniques. *Electronics*, 13(17), 3396. <https://doi.org/10.3390/electronics13173396>
- Lin, T., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017, August 7). *Focal loss for dense object detection*. arXiv.org. <https://arxiv.org/abs/1708.02002>
- Vargas, M. R., Anjos, C. E. M. D., Bichara, G. L. G., & Evsukoff, A. G. (2018). Deep learning for stock market prediction using technical indicators and financial news articles. 2022 *International Joint Conference on Neural Networks (IJCNN)*, 1–8. <https://doi.org/10.1109/ijcnn.2018.8489208>